

Robot Karol

Informatik · Klasse 8 · Arbeitsheft

Inhaltsverzeichnis

01 – Robot Karol kennenlernen	2
Leitfrage	2
Grundbefehle	2
Aufgabe 1 – Vorhersagen	2
Aufgabe 2 – Rechteckweg	2
Aufgabe 3 – Fehleranalyse	2
Aufgabe 4 – Ziegel	2
Programmieren heißt testen	2
Merke	3
02 – Wiederholungen in Robot Karol	4
Rückblick	4
Feste Wiederholung	4
Aufgabe 1	4
Aufgabe 2 – Quadrat	4
Aufgabe 3 – Zieglmuster	4
Verschachtelte Wiederholung	4
Aufgabe 4	4
Aufgabe 5 – Lesbarkeit	4
Merke	4
03 – Bedingungen und Sensoren in Robot Karol	5
Leitfrage	5
Sensoren als Fragen	5
Entscheidung	5
Aufgabe 1	5
Aufgabe 2	5
Bedingte Wiederholung	5
Aufgabe 3	5
Aufgabe 4 – Ziegel suchen	5
Aufgabe 5 – Fehlerfall	6
Merke	6
04 – Unterprogramme und systematisches Problemlösen	7
Rückblick	7
Teile statt eines langen Programms	7
Verbindung zur Objektorientierung	7
Aufgabe 1 – Rechts drehen	7
Aufgabe 2 – Teilprobleme	7
Aufgabe 3 – Wiederverwenden	7
Aufgabe 4 – Planen vor Programmieren	7
Aufgabe 5 – Debugging	7
Merke	8
05 – Karol-Projekt: planen, programmieren, testen	9
Projektauftrag	9
Vorgehen	9
Dokumentation	9
Reflexion	9

01 - Robot Karol kennenlernen

Leitfrage

Wie wird aus einem Algorithmus ein Programm für einen Roboter?

Robot Karol ist eine Lernumgebung, in der ein virtueller Roboter in einer Welt aus Feldern, Wänden und Ziegeln gesteuert wird.

Grundbefehle

Je nach eingesetzter Version heißen die Befehle sinngemäß:

- Schritt
- LinksDrehen
- RechtsDrehen
- Hinlegen
- Aufheben
- MarkeSetzen
- MarkeLöschen

Entscheidend ist: Karol führt Anweisungen genau in der angegebenen Reihenfolge aus.

Aufgabe 1 - Vorhersagen

Beschreibe, wo Karol nach diesem Ablauf steht:

Schritt
Schritt
LinksDrehen
Schritt

Aufgabe 2 - Rechteckweg

Plane einen Ablauf, mit dem Karol ein Rechteck mit Seitenlängen 4 und 2 Feldern abläuft.

Aufgabe 3 - Fehleranalyse

Ein Schüler möchte Karol nach rechts drehen, verwendet aber dreimal LinksDrehen. Funktioniert das? Begründe.

Aufgabe 4 - Ziegel

Plane einen Algorithmus, mit dem Karol drei Ziegel hintereinander ablegt.

Programmieren heißt testen

Nach jeder Änderung:

1. Welt in Ausgangslage bringen.
2. Programm starten.
3. Verhalten beobachten.
4. Fehlerstelle finden.
5. gezielt verändern.

Merke

Ein Programm ist die konkrete Formulierung eines Algorithmus in den Befehlen einer Programmiersprache oder Lernumgebung.

02 - Wiederholungen in Robot Karol

Rückblick

Viele Programme enthalten gleiche Befehle mehrfach.

Leitfrage: Wie können wir Karol kürzere und übersichtlichere Programme geben?

Feste Wiederholung

Statt fünfmal Schritt zu schreiben, verwenden wir eine Wiederholung.

```
wiederhole 5 mal  
    Schritt  
endewiederhole
```

Die genaue Schreibweise kann je nach verwendeter Karol-Version leicht abweichen. Das Prinzip bleibt gleich.

Aufgabe 1

Vereinfache einen Ablauf mit acht aufeinanderfolgenden Schritten.

Aufgabe 2 - Quadrat

Entwickle einen Algorithmus, mit dem Karol ein Quadrat mit Seitenlänge 4 abläuft. Nutze Wiederholungen sinnvoll.

Aufgabe 3 - Ziegemuster

Karol soll fünfmal einen Ziegel ablegen und danach einen Schritt gehen. Formuliere den Algorithmus.

Verschachtelte Wiederholung

Eine Wiederholung kann eine weitere Wiederholung enthalten. Dadurch lassen sich regelmäßige Muster beschreiben.

Aufgabe 4

Plane ein 3 × 4-Muster aus Bewegungen oder Ziegeln. Kennzeichne äußere und innere Wiederholung.

Aufgabe 5 - Lesbarkeit

Vergleiche zwei Programme: eines mit vielen Einzelbefehlen und eines mit Wiederholungen. Welche Vorteile hat die strukturierte Version?

Merke

Wiederholungen reduzieren doppelte Anweisungen und machen regelmäßige Abläufe leichter verständlich und veränderbar.

03 - Bedingungen und Sensoren in Robot Karol

Leitfrage

Wie kann Karol auf seine Umgebung reagieren?

Ein starres Programm funktioniert nur, wenn die Welt genau wie erwartet aufgebaut ist. Bedingungen machen Programme flexibler.

Sensoren als Fragen

Karol kann seine Umgebung prüfen. Je nach Version stehen Abfragen sinngemäß zur Verfügung, zum Beispiel:

- Ist vorne frei?
- Ist links frei?
- Liegt hier ein Ziegel?
- Ist hier eine Marke?

Entscheidung

```
wenn vorne frei dann  
    Schritt  
sonst  
    LinksDrehen  
endewenn
```

Aufgabe 1

Beschreibe in Worten, was der Algorithmus macht.

Aufgabe 2

Karol soll einen Schritt gehen, wenn vor ihm frei ist. Ist dort eine Wand, soll er sich links drehen. Formuliere den Ablauf.

Bedingte Wiederholung

```
solange vorne frei  
    Schritt  
endesolange
```

Damit kann Karol unabhängig von der Länge eines Weges bis zur Wand laufen.

Aufgabe 3

Vergleiche feste Wiederholung und bedingte Wiederholung. Wann ist welche Variante besser?

Aufgabe 4 - Ziegel suchen

Plane einen Algorithmus, der Karol vorwärts laufen lässt, bis er auf einem Feld mit Ziegel steht.

Aufgabe 5 - Fehlerfall

Was passiert, wenn ein Algorithmus voraussetzt, dass irgendwann eine Wand kommt, die Welt aber unendlich bzw. sehr lang ist? Welche Rolle spielt die Endlichkeit eines Algorithmus?

Merke

Bedingungen und Sensoren machen Programme abhängig von der aktuellen Situation statt nur von einer vorher bekannten Welt.

04 - Unterprogramme und systematisches Problemlösen

Rückblick

Bedingungen und Wiederholungen machen Programme flexibler.

Leitfrage: Wie zerlegen wir größere Karol-Probleme in überschaubare Teilaufgaben?

Teile statt eines langen Programms

Wiederkehrende Abläufe können als eigene Unterprogramme bzw. Anweisungsblöcke beschrieben werden.

Beispiele:

- RechtsDrehen
- GeheBisZurWand
- LegeReihe
- BaueSaeule

Ein sinnvoller Name beschreibt, **was** ein Teilprogramm leistet.

Verbindung zur Objektorientierung

Karol ist ein Objekt. Befehle wie Schritt oder selbst definierte Abläufe können als Verhalten des Objekts verstanden werden. Damit verbinden sich Objektorientierung und Algorithmik.

Aufgabe 1 - Rechts drehen

Definiere RechtsDrehen ausschließlich mit vorhandenen Linksdrehungen.

Aufgabe 2 - Teilprobleme

Zerlege die Aufgabe „Karol baut ein Rechteck aus Ziegeln“ in mindestens vier Teilprobleme.

Aufgabe 3 - Wiederverwenden

Welche Vorteile hat ein Unterprogramm, das an mehreren Stellen verwendet wird?

Aufgabe 4 - Planen vor Programmieren

Erstelle für eine Karol-Aufgabe deiner Wahl:

1. Zielbeschreibung,
2. Teilprobleme,
3. benötigte Kontrollstrukturen,
4. mindestens drei Testfälle,
5. erst danach das Programm.

Aufgabe 5 - Debugging

Ordne die Schritte sinnvoll:

- Programm ändern
- Fehler reproduzieren
- erwartetes Verhalten festlegen

- Fehlerstelle eingrenzen
- erneut testen

Merke

Größere Programme werden verständlicher, wenn wir sie in benannte, überschaubare Teilaufgaben zerlegen und systematisch testen.

05 - Karol-Projekt: planen, programmieren, testen

Projektauftrag

Entwickle eine eigene Karol-Welt mit einer klaren Aufgabe. Dein Programm soll mindestens enthalten:

- eine Wiederholung,
- eine Bedingung,
- eine bedingte Wiederholung oder vergleichbare situationsabhängige Lösung,
- mindestens ein sinnvoll benanntes Unterprogramm,
- mindestens drei dokumentierte Testfälle.

Vorgehen

1. Problem beschreiben.
2. Welt skizzieren.
3. Teilprobleme formulieren.
4. Algorithmus in Blöcken planen.
5. Programm umsetzen.
6. mit Testfällen prüfen.
7. Fehler verbessern.
8. Lösung kurz dokumentieren.

Dokumentation

Ziel

Teilprobleme

1.

2.

3.

Verwendete Kontrollstrukturen

Testfälle

Test	Ausgangslage	Erwartung	Ergebnis
1			
2			
3			

Reflexion

Welche Programmstelle war am schwierigsten? Welche Änderung hat die Lösung verbessert?